Figure 3.1 An overview of an ASP.NET Core application. The ASP.NET Core application receives an incoming HTTP request from the browser. Every request passes to the middleware pipeline, which potentially modifies it and then passes it to the endpoint middleware at the end of the pipeline to generate a response. The response passes back through the middleware to the server and finally out to the browser.

Much of this diagram should be familiar to you from figure 1.3 in chapter 1; the request and response and the ASP.NET Core web server are still there. But you'll notice that I've added a reverse proxy to show a common deployment pattern for ASP.NET Core applications. I've also expanded the ASP.NET Core application itself to show the middleware pipeline and the endpoint middleware—the main custom part of your app that goes into generating the response from a request.

The first port of call after the reverse proxy forwards a request is the ASP.NET Core web server, which is the default cross-platform Kestrel server. Kestrel takes the raw incoming network request and uses it to generate an `HttpContext` object that the rest of the application can use.

The HttpContext object

The `HttpContext` constructed by the ASP.NET Core web server is used by the application as a sort of storage box for a single request. Anything that's specific to this particular request and the subsequent response can be associated with it and stored in it, such as properties of the request, request-specific services, data that's been loaded, or errors that have occurred. The web server fills the initial `HttpContext` with details of the original HTTP request and other configuration details and then passes it on to the rest of the application.

NOTE Kestrel isn't the only HTTP server available in ASP.NET Core, but it's the most performant and is cross-platform. I'll refer only to Kestrel throughout the book. A different web server, IIS HTTP Server, is used when running in-